

Data Preprocessing Steps

Viktoria – Accidents_17_23

This dataset underwent rigorous preprocessing to ensure it was ready for modeling, particularly aiming to predict “Accident severity” based on various variables. Each step was carefully chosen to enhance data quality, address inconsistencies, and prepare features for effective predictive modeling.

1. Column Standardization:

- **Configuration:** All column names were converted to capitalized format to maintain consistency and ease of reference.
- **Reasoning:** Standardizing column names helps in ensuring clarity and uniformity across the dataset, facilitating easier data manipulation and analysis.

```
new_accidents = new_accidents.rename(columns=lambda x: x.capitalize())
```

2. Handling Missing Values:

- **Configuration:** Missing values were identified during initial data exploration; however, the dataset did not contain any explicitly missing values across its entirety. Instead, numerous entries were labeled as 'Unknown', reflecting situations where sensor and camera data could not definitively determine categorical values.
- **Reasoning:** No imputation or deletion of missing values was necessary, ensuring the dataset's completeness and alignment with our project's objectives. The found 'Unknown' entries were retained to accurately reflect real-world conditions captured by our data collection methods.

```
missing_values = new_accidents.isnull().sum()
print(missing_values)
```

3. Duplicate Removal:

- **Configuration:** Duplicates were removed using the **drop_duplicates()** method.
- **Reasoning:** Eliminating duplicate records prevents redundancy and ensures that each accident instance is represented only once in the dataset, avoiding skewed results and preventing overfitting during model training.

```
new_accidents.drop_duplicates(inplace=True)
```

4. Feature Selection:

- **Configuration:** The 'Municipality' column was dropped from the dataset.
- **Reasoning:** Since 'Municipality' had uniform data ('Breda'), it was not informative for predicting accident severity and thus removed to streamline the dataset.

```
new_accidents = new_accidents.drop(columns=['Municipality'])
```

5. Value Changes in Variables:

- **Configuration:** In 'First Mode of Transport' and 'Second Mode of Transport', values 'Other vehicle' and '-' were replaced with 'Other'.
- **Reasoning:** This transformation consolidates similar categories ('Other vehicle' and '-') into a single category ('Other'), reducing the complexity of categorical variables and potentially improving model performance.

```
new_accidents['First mode of transport'] = new_accidents['First mode of transport'].replace({'Other vehicle': 'Other', '-': 'Other'})
```

```
new_accidents['Second mode of transport'] = new_accidents['Second mode of transport'].replace({'Other vehicle': 'Other'})
```

6. Data Cleaning:

- **Configuration:** Textual inconsistencies in the 'Town' column were addressed by capitalizing all town names uniformly.
- **Reasoning:** Uniform capitalization minimizes data entry errors and ensures consistency in town names throughout the dataset.

```
new_accidents['Town'] = new_accidents['Town'].str.capitalize()
```

7. Data Type Conversion:

- **Configuration:** Categorical variables were converted to the 'category' data type.
- **Reasoning:** Using categorical data types optimizes memory usage and enhances computational efficiency during modeling and analysis.

```
categorical_cols = ['Accident severity', 'Town', 'First mode of transport', 'Second mode of transport', 'Area type', 'Light condition', 'Road location', 'Road condition', 'Road surface', 'Road situation', 'Speed limit', 'Street', 'Weather']
for col in categorical_cols:
    new_accidents[col] = new_accidents[col].astype('category')
```

8. Outlier Detection and Treatment:

- **Configuration:** Outliers in numeric columns ('Year' and 'Accidents') were identified using the Interquartile Range (IQR) method to ensure robustness of statistical analysis and model performance. However, it was found out that all records with more than one accident in the "Accidents" column were identified as outliers.
- **Reasoning:** Treating outliers ensures that statistical analyses and machine learning models are robust and not unduly influenced by extreme data points. Given the nature of the data and the project's objectives, initially, these outliers were retained. However, upon further analysis, we found that their removal improved model performance and statistical reliability. By excluding these outliers, which represented instances of multiple accidents occurring in close proximity, we streamlined the dataset to better reflect typical scenarios and enhance the accuracy of accident severity predictions. This adjustment ensures our model maintains robustness while focusing on the most representative data points for effective analysis and prediction.

```
# Visualising outliers
import seaborn as sns
sns.boxplot(x=new_accidents['Accidents'])
plt.show()
import numpy as np
# Define a function to detect outliers using IQR
def detect_outliers(df, columns):
    outliers_indices = []
    for col in columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
        outliers_indices.extend(outliers.index)
```

```

        return list(set(outliers_indices))
# Specify columns to check for outliers (numeric columns)
numeric_columns = ['Year', 'Accidents']
# Detect outliers
outlier_indices = detect_outliers(new_accidents, numeric_columns)
print("Rows corresponding to outliers:")
display(new_accidents.loc[outlier_indices].head(20))

```

9. Database Management:

- **Configuration:** Data was stored in a PostgreSQL database table ('new_accidents_preprocessed') after preprocessing.
- **Reasoning:** Storing data in a database facilitates efficient data retrieval, management, and scalability for modeling and future analyses.

```

table_name = 'new_accidents_preprocessed'
schema_name = 'group15_warehouse'

# Drop table if it exists (optional)
print("Attempting to drop existing table if it exists.")
drop_table_query = sql.SQL("DROP TABLE IF EXISTS {}.{};").format(
    sql.Identifier(schema_name), sql.Identifier(table_name)
)
cursor.execute(drop_table_query)
conn_psycopg2.commit()
print("Existing table dropped if it existed.")

# Create table in the specified schema
print("Creating new table.")
create_table_query = sql.SQL('''
    CREATE TABLE {}.{} (
        "Year" INT,
        "Accident severity" VARCHAR(50),
        "Town" VARCHAR(100),
        "First mode of transport" VARCHAR(50),
        "Second mode of transport" VARCHAR(50),
        "Area type" VARCHAR(50),
        "Light condition" VARCHAR(50),
        "Road location" VARCHAR(50),
        "Road condition" VARCHAR(50),
        "Road surface" VARCHAR(50),
        "Road situation" VARCHAR(50),
        "Speed limit" VARCHAR(50),
        "Street" VARCHAR(100),
        "Weather" VARCHAR(50),
        "Accidents" INT
    )
''').format(
    sql.Identifier(schema_name), sql.Identifier(table_name)
)
cursor.execute(create_table_query)
conn_psycopg2.commit()
print("New table created.")

# Prepare data for insertion
print("Preparing data for insertion.")
data_tuples = [tuple(row) for row in new_accidents.values]
print(f"Prepared {len(data_tuples)} rows of data for insertion.")

# Insert DataFrame records into PostgreSQL table
print("Inserting data into the new table.")

```

```

insert_query = sql.SQL("INSERT INTO {}.{} VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)").format(
    sql.Identifier(schema_name), sql.Identifier(table_name)
)
cursor.executemany(insert_query, data_tuples)

# Commit changes
conn_psycopg2.commit()
print("Data inserted successfully.")
print("DataFrame saved to PostgreSQL successfully.")

```

10. Feature Engineering:

- **Configuration:** A new feature 'Road Type' was derived from 'Speed Limit'.
- **Reasoning:** Categorizing road types based on speed limits provides additional contextual information that could potentially influence accident severity prediction, enhancing the model's predictive capabilities.

```

# Define the updated mapping dictionary
speed_to_road_type = {
    '130 km/h': 'Highway',
    '120 km/h': 'Expressway',
    '100 km/h': 'Rural Highway',
    '90 km/h': 'Major Rural Road',
    '80 km/h': 'Secondary Rural Road',
    '70 km/h': 'Minor Rural Road',
    '50 km/h': 'Urban Road',
    '30 km/h': 'Town Center Road',
    '60 km/h': 'Build-up Road',
    '15 km/h': 'Residential Road',
    'Footpace / homezone': 'Honezone Road',
    'Unknown': 'Unknown'
}

# Create the new column 'Road Type' based on 'Speed limit'
new_accidents_preprocessed['Road type'] = new_accidents_preprocessed['Speed limit'].map(speed_to_road_type)

# Check the distribution of the new column
print(new_accidents_preprocessed['Road type'].value_counts())

```

11. Encoding Categorical Variables:

- **Configuration:** Categorical variables were encoded using Label Encoding.
- **Reasoning:** Converting categorical variables into numerical format is essential for machine learning algorithms, which require numerical input. Label Encoding preserves the ordinal nature of categorical variables, ensuring that the model can interpret the order or rank within the data.

```

# Label encode categorical variables
from sklearn.preprocessing import LabelEncoder
categorical_cols = ['Accident severity', 'Town', 'First mode of transport', 'Second mode of transport', 'Area type', 'Light condition', 'Road location', 'Road condition', 'Road surface', 'Road situation', 'Speed limit', 'Street', 'Weather', 'Road type']

label_encoders = {}
for column in categorical_cols:
    le = LabelEncoder()

```

```

new_accidents_preprocessed[column] =
le.fit_transform(new_accidents_preprocessed[column])
label_encoders[column] = le # Store the label encoder for inverse
transformation if needed

```

12. Normalization of Numerical Features:

- **Configuration:** Numerical features ('Year' and 'Accidents') were scaled using StandardScaler.
- **Reasoning:** Standardizing numerical features ensures that each feature contributes equally to model training. This prevents features with larger ranges from dominating the learning process and helps in improving the model's performance and convergence speed.

```

from sklearn.preprocessing import StandardScaler
# Specify numerical features
numerical_cols = ['Year', 'Accidents']
# Initialize StandardScaler for numerical data
scaler = StandardScaler()
# Scale numerical data
new_accidents_preprocessed[numerical_cols] =
scaler.fit_transform(new_accidents_preprocessed[numerical_cols])

```

13. Separating features and target variable:

- **Configuration:** Features (X) were separated from the target variable (y) in the preprocessed dataset.
- **Reasoning:** This separation is a standard preprocessing step to prepare the data for machine learning. By clearly defining the independent variables (features) and the dependent variable (target), it ensures that the model can learn the relationship between them.

```

# Separate features and target variable
X = new_accidents_preprocessed.drop(columns=['Accident severity'])
y = new_accidents_preprocessed['Accident severity']

```

9. Split data into training and testing sets:

- **Configuration:** The dataset was split into training and testing sets using a 70-30 split, with a fixed seed (random_state=42) and stratification applied to y.
- **Reasoning:** Splitting the data ensures that the model can be trained on one subset of data and tested on another, facilitating the evaluation of its performance on unseen data. Using a fixed seed ensures reproducibility, and stratification maintains the class distribution, preventing bias.

```

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.3,
                                                    random_state=42,
                                                    stratify=y)

```

10. Apply SMOTE to balance the classes in the training set:

- **Configuration:** SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training set (X_train and y_train).
- **Reasoning:** Addressing class imbalance by oversampling the minority class helps in preventing the model from being biased towards the majority class. SMOTE generates synthetic samples for the minority class, ensuring that the model has sufficient data to learn from all classes, leading to better generalization and improved performance on the minority class.

```

from imblearn.over_sampling import SMOTE

# Apply SMOTE to balance the classes in the training set
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

```

These preprocessing steps ensure that the dataset is appropriately prepared for modeling, addressing class imbalance and ensuring robust model performance.

Schema diagram of the data warehouse used in the preprocessing steps:

